Attribution-Patching-Guided Replay Refinement for Task-Vector Merging

Anonymous Authors
Draft proposal

July 15, 2026

Abstract

We study the task of merging fine-tuned variants of a shared pretrained model when a small labeled replay buffer for each task is available. After an initial task-arithmetic merge, we refine the merged model with task gradients while using the known task experts as coordinate-wise trust anchors. The resulting update admits two equivalent motivations that we develop in parallel. As a parameter-space analogue of attribution patching, the effect of moving the current model toward a task expert is approximated by a first-order loss attribution, and only coordinates for which the expert direction is locally loss-decreasing are used. As an expert-anchored trust-region replay refinement, the departure from ordinary replay gradient descent is that each coordinate's step is weighted by its distance to the expert, gated to keep only steps that move toward the expert, and clipped so it cannot overshoot—the same coordinates that the attribution view selects. Both perspectives lead to a single refinement algorithm.

1 Related Work and Background

Model merging and weight-space averaging. Model merging combines multiple trained or fine-tuned models into a single model in weight space, avoiding the inference-time overhead of ensembling. Early and influential forms include direct parameter averaging, model soups, and Fisher-weighted merging. Model soups average multiple fine-tuned checkpoints and can improve accuracy without increasing inference cost when the checkpoints lie in a compatible basin (Wortsman et al., 2022). Fisher merging instead weights parameters using Fisher information, motivated by a Laplace approximation to the local posterior around each model (Matena and Raffel, 2022). These approaches motivate the broader view that homologous fine-tuned models can often be combined directly in parameter space, but they do not by themselves resolve interference when the merged models specialize in different downstream tasks.

Task vectors. Task arithmetic represents the effect of fine-tuning as a task vector

$$\tau_t = \theta_t - \theta_0, \tag{1}$$

where θ_0 is a shared pretrained model and θ_t is the model fine-tuned on task t. A merged model is then constructed as

$$\theta_{\text{merge}} = \theta_0 + \sum_{t \in \mathcal{T}} \lambda_t \tau_t, \tag{2}$$

where λ_t are scalar or tensor-wise coefficients. Task Arithmetic demonstrated that such vectors can be added or negated to steer model behavior (Ilharco et al., 2022). This formulation is attractive

because it requires only checkpoints, but naive addition can over-count redundant directions and introduce sign or magnitude conflicts across tasks.

Interference-aware task-vector merging. A major line of work addresses interference between task vectors. TIES-Merging trims small-magnitude updates, resolves sign disagreement, and merges only sign-consistent components (Yadav et al., 2023). DARE randomly drops most delta parameters and rescales the remaining entries, arguing that fine-tuning deltas are highly redundant (Yu et al., 2023). DELLA generalizes this drop-and-rescale idea with magnitude-based sampling (Deep et al., 2024). Model Breadcrumbs similarly builds sparse masks by removing both negligible and outlier perturbations (Davari and Belilovsky, 2023). Localize-and-Stitch explicitly localizes sparse task-relevant regions and stitches only those regions back into the pretrained model (He et al., 2024). These methods share the view that not all task-vector coordinates should be merged equally. The present proposal does not introduce another pruning rule; instead, it asks whether a small amount of supervised replay can repair an initial merge while staying close to the known task experts.

Data-dependent merging baselines. Several merging methods use small amounts of data or model outputs to estimate better combinations. Activated Parameter Locating (APL) estimates task-vector importance through a data-dependent contrast and then uses the result to guide pruning (Kong et al., 2024). RegMean formulates merging as a regression problem that minimizes prediction differences between the merged model and the candidate models (Jin et al., 2023). AdaMerging learns task-wise or layer-wise merge coefficients by minimizing an unsupervised entropy objective on unlabeled samples (Yang et al., 2024). These methods are important baselines because the proposed method also uses data, although it uses labeled replay buffers only for a few post-merge refinement steps rather than for constructing a pre-merge mask or a closed-form fusion rule.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions by gradient-based scores (Syed et al., 2024; Kramar et al., 2024). In parameter space, the analogous approximation is a directional derivative of the task loss along a chosen parameter displacement. Here the relevant intervention is not a replacement of one expert by another. It is the movement from the current merged model toward the expert for task i. This gives a direct bridge between attribution patching and replay-buffer refinement: the attribution score decides which coordinates of the expert direction are locally predicted to decrease the task loss.

Shared and task-specific structure. Several recent methods recognize that task vectors contain both shared and exclusive information. Twin-Merging separates shared and exclusive task knowledge and dynamically combines them at inference time (Lu et al., 2024). Task Vector Bases clusters similar task vectors to compress and reuse shared directions, providing a theoretical and practical framework for reducing redundancy across related tasks (Zeng et al., 2025). FusionBench documents the need for standardized evaluation across model-fusion methods and includes GLUE-style language tasks among its benchmark families (Tang et al., 2024). Our proposal is complementary: rather than interpreting small attribution values as evidence that a coordinate can be pruned, we use the sign of a merge-to-expert attribution only as a local gate for refinement.

Benchmark settings. We use the standard GLUE-8 merging setting as the primary diagnostic benchmark: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE, excluding WNLI as is common in many GLUE model-merging studies. GLUE was introduced as a multi-task benchmark

for natural language understanding (Wang et al., 2018). We use RoBERTa-base as the primary encoder backbone because it is a strong, widely used model for GLUE-style evaluation (Liu et al., 2019). In a full fine-tuning setup, each task has a task-specific classifier or regression head. We therefore treat RoBERTa GLUE-8 as a diagnostic setting for representation merging: the shared encoder is merged and evaluated with the corresponding task-specific head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. Later experiments should include a shared-output setting, for example GLUE or SuperGLUE in a text-to-text T5-style format (Wang et al., 2019; Raffel et al., 2020). A second-modality benchmark is also important: the standard CLIP/ViT task-vector suite with image-classification tasks such as SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, and DTD gives a stronger test of whether the method is specific to GLUE encoders or to task-specific NLP heads (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024). A third standard setting is decoder-only LLM merging, which combines domain-specialized experts—typically instruction following, mathematics, and code—and evaluates each on its own generation benchmark (Yu et al., 2023); MergeBench standardizes this setting with released full-parameter experts and matched fine-tuning and evaluation protocols across the Llama and Gemma families (He et al., 2025). These three settings—CLIP/ViT vision, GLUE and text-to-text NLP, and decoder-only LLMs—constitute the benchmark families used across the recent merging literature, and we position our proposal against all three.

2 Methodology

2.1 Problem Setup

Let $\mathcal{T} = \{1, \ldots, T\}$ denote a set of tasks. All task experts are initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$. Fine-tuning on task t gives expert encoder parameters θ_t and task vector

$$\tau_t = \theta_t - \theta_0. \tag{3}$$

The mergeable encoder parameters are treated as a vector in $\mathbb{R}^d$. We write $r \in \{1, \ldots, d\}$ for a parameter coordinate. Task-specific heads are not merged in the primary GLUE experiments; the merged encoder is evaluated with the corresponding head for each task.

The initial merged encoder is

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^{T} \lambda_i \tau_i, \tag{4}$$

where λ_i are fixed task-arithmetic coefficients, typically uniform or chosen by a small grid search. Let $\mathcal{D}_t^{\text{probe}}$ be a small replay buffer used for refinement, and let $\mathcal{D}_t^{\text{eval}}$ be the evaluation split used only for reporting performance. For each task i, let

$$\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}}) \tag{5}$$

denote the replay-buffer loss obtained by using encoder parameters θ with task i's prediction head. The proposed method does not use a base-relative saliency term and does not construct a pre-merge pruning mask. Its only data-dependent operation is a short replay-buffer refinement of the initial merge.

2.2 Trust-Region Replay Refinement

Starting from the task-arithmetic initialization in Equation (4), the natural data-dependent baseline is ordinary replay-buffer gradient descent, which takes an unconstrained step along the task gradients:

$$\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}}). \tag{6}$$

This baseline treats the merge like any other initialization and lets the replay gradients move each coordinate freely, with nothing tying the refined model to the known task experts. The central departure of our method is precisely this anchoring: rather than taking a free gradient step, every coordinate of the update is tied to the corresponding task expert. For each task i and coordinate r we (i) weight the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so that coordinates already close to the expert barely move while coordinates far from it move more; (ii) keep the step only where it moves the model *toward* the expert, discarding coordinates whose replay gradient would pull away from it; and (iii) clip the resulting step to the expert-distance trust region so that a single update can never overshoot the expert. These three operations—expert-distance weighting, the expert-agreement gate, and the trust-region clip—are what distinguish the refinement from ordinary replay gradient descent, and together they keep the whole refinement anchored to the known experts rather than letting it drift.

A secondary and comparatively minor design choice concerns how the per-task updates are scheduled within a refinement step. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task's update immediately after computing it, so that its gradient, expert direction, and gate are evaluated at the same model state to which it is applied. Let $\theta^{(s,0)} = \theta^{(s)}$, and let π_s be the task order for sweep s; π_s can be fixed, cyclically rotated, or randomly permuted. For within-sweep index $k \in \{1, \dots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \tag{7}$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}, \tag{8}$$

$$m_{i,r}^{(s,k)} = \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}, \tag{9}$$

$$\widetilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}. \tag{10}$$

From this optimization side the coordinate mask $m_{i,r}^{(s,k)}$ arises as a trust-region acceptance rule rather than as an attribution score: we keep only those coordinates whose replay-gradient step also moves the current model toward expert i, and discard the coordinates where the replay step would pull away from the expert and out of the expert-anchored trust region. This is the same condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ that Section 2.3 obtained from the attribution-patching sign in Equation (19), now read as *stay within the expert-anchored trust region* instead of *move along a locally loss-decreasing intervention*. The two perspectives therefore agree coordinate by coordinate. The raw update is first scaled by the learning rate η and *then* clipped by the the current distance to the expert, so that the applied per-coordinate step is bounded by $|v_{i,r}^{(s,k)}|$ regardless of η,

$$u_{i,r}^{(s,k)} = \text{clip}\Big(\eta\, \widetilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|\Big), \tag{11}$$

and the current model is immediately updated with this clipped step:

$$\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}. \tag{12}$$

After all tasks in the sweep have been processed, we set

$$\theta^{(s+1)} = \theta^{(s,T)}. \tag{13}$$

The robustness of the refinement comes primarily from the expert anchoring rather than from this scheduling: the distance weighting and gate keep every coordinate pointed at its expert, and the trust-region clip caps how far it can travel. The immediate application of each update, though secondary to the anchoring, has two motivations of its own. First, each update is applied at the same local model state on which its gradient, expert direction, and gate were computed. Second, the trust-region clipping in Equation (11) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step. Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η; for ≤ 1 a single update can never overshoot the expert in a coordinate, so the refinement stays within the expert-distance trust region and cannot diverge no matter how large η is. The learning rate then controls only how many coordinates are driven to the trust-region boundary, not the maximum per-coordinate step size. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j. We therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update; the defining operation remains the coordinate gate in Equation (19), whether it is read as an attribution-patching sign or as a trust-region acceptance rule. Under either reading the procedure is the single algorithm summarized in Algorithm 1.

Algorithm 1 Sequential expert-direction-gated replay refinement

Require: Pretrained encoder θ_0; task experts $\{\theta_i\}_{i=1}^T$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^T$; task-arithmetic weights $\{\lambda_i\}_{i=1}^T$; steps S; learning rate η; gate threshold ϵ_{gate}.

$\tau_i \leftarrow \theta_i - \theta_0$ for all tasks i.
$\theta^{(0)} \leftarrow \theta_0 + \sum_{i=1}^T \lambda_i \tau_i$. ▷ or the calibrated merge $\theta_{\text{cal}}^{(0)}$, Eq. (21) (two-stage variant)
for $s = 0, \ldots, S-1$ **do**
  $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
  Choose a task order π_s over $\{1, \ldots, T\}$.
  for $k = 1, \ldots, T$ **do**
    $i \leftarrow \pi_s(k)$.
    $g_i^{(s,k)} \leftarrow \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i's head.
    $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
    for each mergeable coordinate r **do**
      $m_{i,r}^{(s,k)} \leftarrow \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}$.
      $\widetilde{u}_{i,r}^{(s,k)} \leftarrow -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}$.
      $u_{i,r}^{(s,k)} \leftarrow \text{clip}(\eta\, \widetilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|)$. ▷ clip after learning-rate scaling
    end for
    $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
  end for
  $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
end for
return $\theta^{(S)}$.

2.3 A Second Perspective: Attribution Patching in Parameter Space

The refinement procedure developed in the previous section (Algorithm 1) can be motivated from a second independent starting point that arrive at the same per-coordinate update, comin fron an interpretability-style argument: it treats each move toward a task expert as a parameter-space intervention and uses a first-order attribution-patching score to decide, coordinate by coordinate, whether that intervention is locally loss-decreasing.

The attribution step is a local calculation around the current model state. To make the sequential update rule explicit, let ϑ denote the current merged encoder immediately before applying a single task update. For task i, the task expert θ_i is a known reference solution, and the merge-to-expert direction is

$$v_i(\vartheta) = \theta_i - \vartheta. \tag{14}$$

The corresponding parameter-space intervention is the path

$$\Gamma_i(\alpha; \vartheta) = \vartheta + \alpha v_i(\vartheta), \qquad \alpha \in [0, 1], \tag{15}$$

where $\alpha = 0$ is the current model and $\alpha = 1$ is the task expert. Let

$$g_i(\vartheta) = \nabla_\theta \mathcal{L}_i(\vartheta; \mathcal{D}_i^{\text{probe}}) \tag{16}$$

be the task gradient at the current model, evaluated with task i's head. A first-order attribution-patching approximation to the loss change caused by moving from ϑ toward expert i is

$$\Delta_i^{\text{AP}}(\vartheta) = \langle g_i(\vartheta), v_i(\vartheta) \rangle = \sum_{r=1}^{d} a_{i,r}^{\text{AP}}(\vartheta), \tag{17}$$

with coordinate-wise contribution

$$a_{i,r}^{\text{AP}}(\vartheta) = g_{i,r}(\vartheta) v_{i,r}(\vartheta). \tag{18}$$

A negative contribution means that moving coordinate r toward the task expert is locally predicted to reduce task i's loss. A positive contribution means that the same movement is locally predicted to increase the loss. Thus the loss-decreasing expert gate is

$$m_{i,r}(\vartheta) = \mathbf{1}\Big\{ a_{i,r}^{\text{AP}}(\vartheta) < -\epsilon_{\text{gate}} \Big\}, \tag{19}$$

where $\epsilon_{\text{gate}} \geq 0$ is an optional confidence threshold; the default method uses $\epsilon_{\text{gate}} = 0$. The condition $g_{i,r}(\vartheta) v_{i,r}(\vartheta) < 0$ is exactly the condition that the expert direction agrees with the negative gradient in coordinate r.

The resulting coordinate-wise update for task i is a distance-scaled negative-gradient step, masked by the attribution-patching sign:

$$\widetilde{u}_{i,r}(\vartheta) = -g_{i,r}(\vartheta) \; |v_{i,r}(\vartheta)| \; m_{i,r}(\vartheta). \tag{20}$$

Whenever $m_{i,r}(\vartheta) = 1$, this update points toward the expert because $-g_{i,r}(\vartheta)$ and $v_{i,r}(\vartheta)$ have the same sign. The factor $|v_{i,r}(\vartheta)|$ makes the step small when the current model is already close to the expert in that coordinate and larger when the coordinate is far from the expert. Importantly, the gate only certifies a local first-order effect for task i at the current model state. It does not by itself prove that the coordinate is harmless for other tasks, so cross-task interference must be measured experimentally.

2.4 Calibrated Initialization and the Two-Stage Method

The initialization in Equation (4) applies a single scalar λ_i to task i's entire task vector, the coarsest possible reweighting of the merge. When the same labeled replay buffers are available, the coefficient can instead be resolved at the granularity of individual parameter tensors, which turns the fixed initialization into a short first stage of the method. Partition the d mergeable coordinates into B disjoint blocks $\{\mathcal{B}_b\}_{b=1}^{B}$, one per parameter tensor, and let P_b be the diagonal projector onto block b, so that $\sum_b P_b = I$ and $\tau_i = \sum_b P_b \tau_i$. The *calibrated merge* assigns a separate coefficient to each (task, block) pair,

$$\theta_\lambda^{(0)} = \theta_0 + \sum_{i=1}^{T}\sum_{b=1}^{B} \lambda_i^{(b)} P_b \tau_i, \qquad \lambda \in \mathbb{R}^{T\times B}, \tag{21}$$

which recovers the task-arithmetic merge of Equation (4) when $\lambda_i^{(b)} = \lambda_i$ for all b, and uniform averaging when $\lambda_i^{(b)} = 1/T$. The coefficients are fit on the replay buffers by minimizing the supervised task loss with an ℓ_2 pull toward an initial value λ_0,

$$J(\lambda) = \sum_{i=1}^{T} \mathcal{L}_i\Big(\theta_\lambda^{(0)}; \mathcal{D}_i^{\text{probe}}\Big) + \tfrac{\rho}{2}\sum_{i,b}\big(\lambda_i^{(b)} - \lambda_0\big)^2, \tag{22}$$

by gradient descent from $\lambda_i^{(b)} = \lambda_0$, with $\rho \geq 0$ and early stopping on a held-out portion of each buffer; both guard the $T \times B$ coefficients against overfitting the small buffer.

The fit is inexpensive because $\theta_\lambda^{(0)}$ is *linear* in λ. The gradient of the data term with respect to one block coefficient is a directional derivative of the summed task loss along the corresponding block-projected task vector,

$$\frac{\partial}{\partial \lambda_j^{(b)}} \sum_{i=1}^{T} \mathcal{L}_i\Big(\theta_\lambda^{(0)}\Big) = \left\langle \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i\Big(\theta_\lambda^{(0)}; \mathcal{D}_i^{\text{probe}}\Big),\ P_b \tau_j \right\rangle, \tag{23}$$

so all $T \times B$ coefficient gradients follow from one backward pass per task—the same task gradients the refinement already computes—contracted with the task vectors block by block. No computational graph is retained through λ, and the memory cost is $O(d)$ rather than the $O(Td)$ of differentiating the explicit sum. Write $\theta_{\text{cal}}^{(0)}$ for the resulting merge.

The two-stage method uses $\theta_{\text{cal}}^{(0)}$ in place of $\theta^{(0)}$ as the starting point of Algorithm 1; the refinement is otherwise unchanged, and in particular its expert directions $v_i = \theta_i - \theta$ still anchor to the same experts $\{\theta_i\}$. The stages address complementary error scales. Calibration is a coarse reweighting in the low-dimensional, well-conditioned space of $T \times B \sim 10^3$ tensor coefficients: it corrects the per-tensor *mixing* of the merge but cannot change the pattern of coordinates within a tensor. The refinement is a fine, per-coordinate correction in the full $d \sim 10^8$-dimensional space: it repairs the residual cross-task *interference* that a global per-tensor scale cannot express. Because the objective in Equation (22) is the supervised task loss rather than an unlabeled confidence surrogate, calibration cannot be minimized by making the merged model confidently wrong, and it therefore applies whether the task heads are frozen zero-shot maps or trained classifiers—a distinction that matters empirically and that we return to in Section 4.

3 Experimental Protocol

Primary diagnostic setting: RoBERTa GLUE-8. The primary benchmark is GLUE-8: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE. We use RoBERTa-base as the default encoder.

Each task expert is obtained by fine-tuning from the same pretrained checkpoint with matched hyperparameter budgets. The primary setting merges full fine-tuned encoders and keeps task-specific heads fixed for evaluation. This setting is intentionally described as *oracle-task-id, multi-head encoder merging*; it is a useful and inexpensive diagnostic but not a complete demonstration of single-model deployment.

Recommended proof-of-concept order. The first experiments should be deliberately smaller than full GLUE-8, but the final report should avoid relying only on hand-picked related tasks.

1. **Code smoke test:** use a distilled RoBERTa-style encoder (Sanh et al., 2019) on MRPC and STS-B. This pair is small enough for rapid iteration and tests whether the implementation can exploit related sentence-pair tasks.

2. **First convincing three-task result:** use RoBERTa-base on MRPC, STS-B, and SST-2. MRPC and STS-B form a related sentence-pair pair, while SST-2 acts as a less-related single-sentence control task.

3. **Systematic pairwise GLUE study:** evaluate all pairwise merges among the eight GLUE tasks and report a heatmap of mean retention, worst-task retention, and expert-relative drop. This guards against cherry-picking a favorable task subset.

4. **Structured GLUE subsets:** evaluate related subsets (MRPC/STS-B and MRPC/QQP/STS-B), a mixed-format subset (STS-B/SST-2/QNLI), small-task stress subsets (RTE/CoLA/MRPC and QQP/RTE/CoLA), and the entailment cluster (MNLI/QNLI/RTE).

5. **Full GLUE-8 scale-up:** run the final method and all feasible baselines on all eight tasks. CoLA and RTE require multiple seeds because they are small or high-variance; MNLI requires explicit compute budgeting because it is much larger than the other tasks.

The main paper-aligned result should use RoBERTa-base. Distilled models are useful for debugging hyperparameters and implementation errors but should not be the only evidence.

Secondary benchmark settings. To address the limitations of multi-head GLUE, we include three secondary benchmark tracks once the RoBERTa proof of concept is stable. First, a shared-output NLP track evaluates GLUE or SuperGLUE in a text-to-text format using T5-style models, so that the merged model uses a common generation head rather than task-specific classifiers (Wang et al., 2019; Raffel et al., 2020). Second, a CLIP/ViT vision track evaluates the standard task-vector image-classification suite: SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, and DTD (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024). The vision track is especially valuable because it tests a different modality, different loss geometry, and stronger domain shifts than GLUE.

Third, a decoder-only LLM track tests whether the refinement extends beyond encoders to generative models, which is where much of the recent merging literature is concentrated. We adopt the MergeBench setting (He et al., 2025), which releases full-parameter domain experts—instruction following, mathematics, multilingual understanding, coding, and safety—for the Llama and Gemma families together with standardized fine-tuning and evaluation protocols and a suite of merging baselines. Because these experts are full fine-tunes sharing a common pretrained checkpoint, their task vectors and merge-to-expert directions are well defined, so the method applies unchanged; and because the checkpoints are published, no expert training is required on our side, which keeps the comparison to prior merging methods fair and reproducible. We begin at a small full-fine-tuning-feasible scale (a 2–3B base model such as Llama-3.2-3B or Gemma-2-2B) and focus

on the instruction/mathematics/code triple, since these experts are known to interfere strongly and thus stress worst-task retention—the regime in which the gate helped most on CLIP/ViT. Following common practice, we prefer judge-free automatic metrics where possible (for example verifiable instruction-following constraints, exact-match on grade-school mathematics, and functional-correctness pass rates on code). Generation-based evaluation, rather than expert training, is the main cost in this track; we therefore subsample evaluation sets during hyperparameter search and report full-set numbers only for final configurations. This decoder track should complement, not replace, the controlled RoBERTa and CLIP/ViT studies.

Replay data. For refinement, we use $n \in \{4, 8, 16, 32, 64, 128\}$ replay examples per task. Replay examples are sampled from training data or from a held-out split separated from the final validation split. For classification tasks, replay buffers should be class-balanced whenever possible; for imbalanced tasks such as QQP and for small tasks such as RTE, we report the exact sampling procedure. We report sensitivity to the number of replay examples, variation across replay samples, and the area under the replay-budget curve rather than only the best tuned replay size.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by data regime.

Checkpoint-only baselines.
Pretrained encoder, individual single-task experts, uniform model averaging, task arithmetic, Fisher merging, TIES, DARE, DELLA-style magnitude sampling, Model Breadcrumbs, and Localize-and-Stitch if implementation allows.

Data-dependent but label-free baselines.
RegMean, AdaMerging, APL, and any FusionBench-compatible unsupervised or activation-based method that can use the same number of unlabeled examples.

Labeled-replay baselines.
Ordinary replay-buffer gradient descent from the same task-arithmetic initialization; distance-scaled gradient descent without the attribution gate; head-only replay calibration; encoder-only replay fine-tuning; encoder-plus-head replay fine-tuning; multitask replay fine-tuning from the pretrained checkpoint; and multitask replay fine-tuning from the task-arithmetic merge.

Sanity controls.
Inverted gate, random gate with matched density, wrong-expert directions, shuffled replay labels, frozen first-step gates, and the aggregated-U version of the method. The aggregated-U ablation is important because the proposed algorithm applies each task update immediately after it is computed.

All labeled-replay baselines should use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget. This is essential because the strongest alternative explanation for a positive result is simply that the method performs supervised post-merge replay fine-tuning.

Metrics. For each GLUE task, we report the standard validation metric: Matthews correlation for CoLA, accuracy for SST-2, matched/mismatched accuracy for MNLI, correlation for STS-B, and accuracy/F1-style scores for the paraphrase and question-pair tasks according to the GLUE convention. Raw task scores, expert scores, pretrained scores, task-arithmetic scores, and absolute

drops from the corresponding expert are primary. Because correlation-style metrics can be near zero or negative, the normalized retention metric is

$$\mathrm{NormRet}_t = \frac{\mathrm{Score}_t(\theta_{\mathrm{merge}}) - \mathrm{Score}_t(\theta_0)}{\mathrm{Score}_t(\theta_t) - \mathrm{Score}_t(\theta_0) + \epsilon_{\mathrm{metric}}}, \tag{24}$$

where $\epsilon_{\mathrm{metric}}$ prevents division by very small denominators. We report mean normalized retention, worst-task normalized retention, harmonic mean normalized retention, and expert-relative drop. Worst-task retention is important because a merge can improve average score while catastrophically harming a small task such as RTE or CoLA. For each main table, we also report confidence intervals across expert fine-tuning seeds and replay-buffer seeds.

Task-similarity and interference analysis. We group GLUE-8 tasks into interpretable clusters. The paraphrase and similarity cluster contains $\{\mathrm{MRPC}, \mathrm{QQP}, \mathrm{STS\text{-}B}\}$; the entailment-style cluster contains $\{\mathrm{MNLI}, \mathrm{QNLI}, \mathrm{RTE}\}$; and the single-sentence cluster contains $\{\mathrm{CoLA}, \mathrm{SST\text{-}2}\}$. We test whether expert-direction-gated refinement helps most in related task clusters, where movement toward one expert is more likely to be compatible with other tasks. We also compute task-vector cosine similarity, cross-task transfer performance, average attribution-patching sign agreement, and the cross-task interference matrix

$$C_{j \leftarrow i}^{(s,k)} = \left\langle \nabla_\theta \mathcal{L}_j(\theta^{(s,k-1)}; \mathcal{D}_j^{\mathrm{probe}}), u_i^{(s,k)} \right\rangle. \tag{25}$$

Negative entries indicate that task i's update is locally predicted to help task j, while positive entries indicate predicted interference. This matrix directly tests whether the gate is only self-protective or also tends to align across related tasks.

3.1 Ablations and Diagnostics

We plan the following ablations:

1. **Gate sign and necessity:** compare the proposed gate $g_{i,r} v_{i,r} < 0$ with no gate, an inverted gate $g_{i,r} v_{i,r} > 0$, and a random mask of the same density.

2. **Gate confidence and granularity:** vary ϵ_{gate}, compare coordinate-wise gates with tensor-wise or block-wise gates, evaluate top-k most negative attribution masks, and test sign-stability masks formed from multiple replay minibatches.

3. **Sequential update rule:** compare the proposed immediate single-task update with the aggregated-U variant, fixed task order, cyclic task order, and random task order.

4. **Distance scaling:** compare ordinary replay-gradient descent, distance-scaled gradient descent without the gate, direct expert-interpolation steps $u_i \propto m_i v_i$, and the full gated update.

5. **Clipping and normalization:** compare no normalization, tensor-wise RMS normalization, per-task global norm clipping, and post-update validation line search.

6. **Replay budget and variance:** vary the number of replay examples per task and report variation across replay samples and random seeds.

7. **Number of refinement steps:** compare $S \in \{0, 1, 2, 5, 10\}$ to test whether most gains occur in the first few steps or whether overfitting appears.

8. **Expansion point:** recompute gradients and gates before every single-task update versus freezing the first-sweep gates from $\theta^{(0)}$.

9. **Label and expert sanity checks:** compare correct labels with shuffled replay labels, and correct expert directions with wrong-expert directions.

3.2 Hypotheses

The project evaluates five hypotheses:

H1: Attribution-patching signs predict useful expert directions.
Coordinates or blocks with $g_{i,r}v_{i,r} < 0$ should be more likely to reduce task i's loss when moved toward expert i than coordinates or blocks with $g_{i,r}v_{i,r} > 0$.

H2: Expert-guided replay refinement improves task arithmetic.
Starting from the same task-arithmetic initialization and using the same replay budget, the gated update should outperform ordinary replay-buffer gradient descent and distance-scaled gradient descent without the gate.

H3: Sequential single-task updates improve worst-task retention.
Applying each clipped task update immediately after it is computed should reduce overshoot and stale-gradient effects relative to an aggregated-U update, improving worst-task retention under the same replay budget.

H4: Benefits are larger for related tasks.
The method should help most in related task clusters, especially MRPC, QQP, and STS-B, where expert directions are more likely to be mutually compatible.

H5: Gains are not GLUE-specific.
A positive GLUE result should transfer at least partially to a shared-head text-to-text setting, to the CLIP/ViT vision task-vector suite, or to decoder-only LLM merging in the MergeBench setting.

4 Preliminary Results

We report initial experiments in the RoBERTa-base multi-head setting (oracle task identity at evaluation). Unless noted, refinement uses S sweeps from the task-arithmetic merge with the corrected trust region applied *after* the learning rate (Eq. (11)) and threshold $\epsilon_{\text{gate}} = 0$. Each method family (ordinary replay gradient descent, the proposed gated update, and the ungated distance-scaled update) is given its own learning-rate search of equal size; we report the best configuration per family by mean normalized retention (Eq. (24)). Section 4.1 additionally compares the refinement against interference-aware and label-free merge baselines, and composes it with them.

These results are preliminary in three respects that we state explicitly. First, the hyperparameter-sweep cells are single-seed, and individual cells carry a run-to-run spread of roughly ± 0.02 mean retention (Table 2 and the discussion of learning-rate sensitivity below), so sweep-level differences smaller than that should not be read as orderings. The headline configurations, however, are replicated over five replay-buffer seeds (Table 6), and we flag explicitly the one earlier single-seed conclusion that did not survive replication. Second, hyperparameters (learning rate, and in Section 4.1 also λ, sparsity, and density) are selected *per family* by mean normalized retention on the evaluation split. This is a uniform and therefore fair rule across families, but it is oracle

Table 1: Three-task RoBERTa-base proof of concept (MRPC, STS-B, SST-2; $n = 64$, $S = 5$). Mean normalized retention; best learning rate per family. Left: best configuration, mean ± std over five seeds. Right: a single-seed contrast at a large fixed learning rate, isolating the gate's effect.

Method	Best mean NormRet (± std)	at $\eta = 64$ (1 seed)
Task-arithmetic merge	0.558	—
Ordinary replay GD	0.831 ± 0.005	—
Ungated distance-scaled (no gate)	0.835 ± 0.017	-1.07 (diverges)
APR (gated, proposed)	$\mathbf{0.840 \pm 0.021}$	**0.849** (stable)

selection: a stricter protocol, in which every method that consumes labeled replay data selects its hyperparameters on that replay buffer alone and touches the evaluation split once, is the subject of ongoing experiments. Third, the multi-head GLUE setting assumes oracle task identity.

Three-task proof of concept. On the MRPC/STS-B/SST-2 triple ($n = 64$ replay examples per task, $S = 5$), all refinement methods recover most of the gap from the task-arithmetic merge (mean normalized retention 0.558) toward the experts, and the three families are statistically tied at their tuned optima (Table 1). The decisive difference is robustness to the learning rate: the gated update attains its best score at a much larger step ($\eta = 64$) and remains stable there, whereas the ungated update—identical except for the gate—*diverges* at the same learning rate (mean normalized retention -1.07). The attribution gate thus acts as a protective mask that enables large, safe steps by suppressing the locally loss-increasing expert directions.

GLUE-8 scale-up. Merging all eight experts (one consistent RoBERTa-base fine-tuning source) is a much harder, more interference-prone setting: the task-arithmetic merge attains only mean normalized retention 0.320, with MRPC driven below the pretrained baseline. Here the value of expert anchoring grows: both anchored updates clearly outperform ordinary replay gradient descent, by a larger margin than in the three-task case and especially on worst-task retention (Table 2). The gate is approximately performance-neutral relative to the ungated anchored update on mean retention—the ordering flips between the two budgets and the gaps are within single-seed noise—but it again provides a wider safe learning-rate range (the gated update is stable through $\eta = 16$, while the ungated update peaks at $\eta = 2$–4 and diverges by $\eta \gtrsim 8$–16). MRPC and CoLA remain the persistent worst tasks; QQP, STS-B, MNLI, and QNLI recover to 0.78–0.86.

A caveat on the boundary of that safe range. At $\eta = 32$ the gated update attained 0.614 (worst 0.142) in one run, but an identically configured re-run collapsed to -0.093 (worst -3.467, MRPC), so we report the stable $\eta = 16$ configuration instead. The replay gradients are deterministic (they are computed in `eval` mode, so no dropout noise enters), which localises the cause: floating-point non-associativity on the GPU—the embedding backward pass accumulates with atomics, and library kernel selection varies with machine state—perturbs the gradient at the level of its last bits. The gate then *discretises* that perturbation, because coordinates with $g_{i,r}v_{i,r} \approx 0$ can flip sign, and near the divergence boundary the resulting trajectories separate. This is a property of the method worth stating plainly: the trust region guarantees that large steps remain *bounded* (Section 2.2), but near the critical learning rate it does not make the outcome *reproducible*. Reported learning rates should therefore be interior to the stable range, not at its edge.

Table 2: GLUE-8 (all eight tasks, RoBERTa-base, single seed). Best mean normalized retention per family (worst-task retention in parentheses); the task-arithmetic merge baseline is 0.320 (worst −0.175). APR is reported at its largest stable learning rate, $\eta = 16$; the $\eta = 32$ configuration reached 0.614 once but is not reproducible (see text).

Method	n=64, S=5	n=32, S=10
Ordinary replay GD	0.574 (0.04)	0.588 (0.04)
Ungated distance-scaled (no gate)	**0.637** (0.15)	0.607 (0.05)
APR (gated, proposed)	0.595 (0.11)	**0.617** (0.05)

What the attribution gate is and is not doing. Two diagnostics temper the interpretation of the gate. First, the gate keeps a strikingly stable $\approx 36\%$ of coordinates across sweeps, tasks, and learning rates. A control attributes most of this to gradient sparsity rather than to the expert geometry: roughly 31% of encoder coordinates are word-embedding rows for tokens absent from the small replay buffer and therefore have exactly zero gradient (always masked), and a random direction of matched magnitudes reproduces almost the same density (0.345 vs. 0.361 for the true expert direction). The expert direction itself contributes only a small signal beyond chance. Second, the trust-region clip is almost never active at useful learning rates ($< 0.1\%$ of gated coordinates), functioning as a rare-event safeguard against the few large-gradient coordinates rather than as a routine operation. The gate's measurable benefit is therefore concentrated in learning-rate robustness—masking the loss-increasing active coordinates so that large steps remain safe—rather than in a higher peak score at a tuned learning rate.

A random mask of matched density makes this concrete. Over five replay-buffer seeds on GLUE-8 (Table 6), the true gate (0.542 ± 0.062) is statistically indistinguishable from—indeed nominally worse than—a random gate of the same density (0.568 ± 0.031), and both trail the ungated anchored update (0.593 ± 0.040). On this modality the gate's *sign* carries no measurable signal beyond its density, exactly as the embedding-sparsity account predicts. The same control behaves in the opposite way on CLIP (Section 4.1), which sharpens the modality split into a clean statement: the attribution sign is informative where gradients are dense, and vacuous where the replay buffer leaves most coordinates without gradient.

CLIP/ViT vision track. We next test whether the method transfers beyond GLUE encoders to the standard eight-task CLIP task-vector suite (SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD) with a CLIP ViT-B/32 backbone. Here the mergeable model is the image encoder, and each task keeps a *frozen* zero-shot classification head derived from the CLIP text encoder over class-name prompts; the experts are publicly available fine-tuned image encoders, merged with uniform $\lambda_i = 0.3$ (a value we later show is over-scaled; see Section 4.1). Refinement uses $n = 64$, $S = 5$, and the trust region of Eq. (11) with $\gamma = 1$ (single seed). This is a stronger test than GLUE: a different modality, dense gradients without the word-embedding sparsity discussed above, and larger domain shifts. The task-arithmetic merge is weak at this coefficient (mean normalized retention 0.270, worst-task −0.527; SUN397 and Cars are pushed *below* the zero-shot backbone). All three families recover substantial ground, but—in contrast to GLUE-8—the attribution gate attains both the best mean retention and, *among the three replay-refinement families*, the only positive worst-task retention (Table 3). This last property is specific to that comparison and does not extend to merge methods generally: tuned task arithmetic, Model Breadcrumbs, and TIES all keep worst-task retention positive without using any data at all (Section 4.1). The gated update

Table 3: Eight-task CLIP ViT-B/32 suite (single seed, $n = 64$, $S = 5$, $\gamma = 1$, uniform $\lambda_i = 0.3$). Best learning rate per family by mean normalized retention (Eq. (24)); worst-task retention in parentheses. The gated update is best on both, and is the only method keeping worst-task retention positive.

Method	Mean NormRet	Worst-task
Task-arithmetic merge	0.270	−0.527
Ordinary replay GD	0.458	−0.203
Ungated distance-scaled (no gate)	0.536	−0.086
APR (gated, proposed)	**0.598**	**+0.159**
Inverted-gate control	−1.682	−5.386

Table 4: Per-task top-1 accuracy on the eight-task CLIP suite: zero-shot backbone, task expert, task-arithmetic merge, and the proposed gated refinement (APR, $\eta = 8$). Refinement recovers most of the merge's interference loss, including on Cars and SUN397, which the merge had degraded below the zero-shot backbone.

	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD	Avg.
Zero-shot	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439	0.478
Expert	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794	0.903
Merge	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472	0.673
APR	**0.650**	**0.648**	**0.769**	**0.923**	**0.874**	**0.851**	0.961	**0.578**	**0.782**

improves the merge from 0.270 to 0.598 mean and from −0.527 to +0.159 worst-task, beating ordinary replay gradient descent by +0.14 mean / +0.36 worst and the ungated anchored update by +0.06 mean / +0.25 worst. The gains concentrate on the most interference-damaged tasks: Cars and SUN397, which the merge drove below the zero-shot floor, are precisely the ones the gate rescues (Table 4), whereas the ungated update sacrifices Cars catastrophically at its larger steps (worst-task −2.97 at $\eta = 8$). An inverted-gate control diverges (mean −1.68), confirming that the sign of the attribution product carries the signal. Error bars and a second control now make this solid (Table 6): over five replay-buffer seeds APR attains 0.605 ± 0.008, against 0.547 ± 0.007 for the ungated update and 0.460 ± 0.002 for ordinary replay GD, while a *random* gate of matched density collapses to 0.498 ± 0.117 with wildly unstable worst-task retention (-0.564 ± 0.888). On CLIP the attribution sign is the signal; matching its density with random coordinates reproduces none of the benefit. A smaller three-task vision study (EuroSAT/GTSRB/MNIST, single seed) shows the same ordering more sharply: APR 0.940 mean / 0.915 worst, versus 0.917/0.876 for the ungated update, 0.874/0.792 for ordinary replay GD, and 0.811/0.721 for the merge. As in GLUE, the best learning rate falls as tasks are added (the gated update peaks at $\eta = 32$ on three tasks and at $\eta = 8$ on eight). Together these results support H5—the method transfers to a second modality—and indicate that the gate's usefulness is *modality-dependent*: approximately neutral on GLUE encoders but beneficial, on both mean and worst-task retention, on CLIP/ViT.

Replay budget and refinement horizon (CLIP/ViT). Two further studies on the eight-task suite probe the labeled-replay budget and the number of refinement sweeps, comparing the gated update directly against ordinary replay gradient descent—the strongest baseline that also uses labeled replay. First, reducing the replay buffer from $n = 64$ to 32 and 16 examples per task degrades both methods gracefully, but the gated update retains an almost constant advantage of

Table 5: Replay-budget study on the eight-task CLIP suite (mean normalized retention; worst-task in parentheses). Best configuration per family; the gated update (APR) near-converges by $S = 5$–10 sweeps, while ordinary replay GD is given a longer horizon ($S = 25$–35) as a generous baseline. APR holds an $\approx +0.10$ mean advantage at every budget and is the only method with non-negative worst-task retention down to $n = 16$.

Replay budget	**APR (ours)**	Ordinary replay GD
$n = 16$	**0.522** (+**0.018**)	0.427 (−0.266)
$n = 32$	**0.570** (+**0.122**)	0.476 (−0.197)
$n = 64$	**0.598** (+**0.159**)	0.526 (−0.135)

$\approx +0.10$ mean normalized retention at every budget and remains the only method with non-negative worst-task retention down to $n = 16$ (Table 5). Its best learning rate also decreases as the buffer shrinks (from $\eta = 8$ at $n = 64$ to $\eta = 6$ at $n \leq 32$), consistent with noisier replay gradients favoring smaller trust-region steps. Second, on the refinement horizon: the gated update is essentially converged after five sweeps (mean retention $0.598 \to 0.615$ from $S = 5$ to $S = 15$ at $n = 64$, with no sign of overfitting the buffer), whereas ordinary gradient descent is still improving at $S = 35$ ($0.458 \to 0.504 \to 0.526$ at $S = 5, 15, 35$) yet does not catch up and—critically—never lifts its worst task (SUN397) back above the zero-shot backbone. Thus the expert anchor, rather than additional replay data or gradient steps, is what recovers the most interference-damaged tasks.

4.1 Merge baselines, and composition with them

Every experiment above starts from a uniform task-arithmetic merge, which leaves the central objection unanswered: perhaps the refinement is only repairing a weak initialization, and a better *merge* would make it unnecessary. We therefore evaluate the refinement against, and on top of, the merge methods it is supposed to complement (Table 7). Four checkpoint-only baselines use no data at all: task arithmetic with a tuned coefficient, TIES (Yadav et al., 2023), DARE (Yu et al., 2023), their composition DARE-TIES, and Model Breadcrumbs (Davari and Belilovsky, 2023). Two label-free but data-dependent baselines consume unlabeled inputs: AdaMerging (Yang et al., 2024), which learns task-wise or layer-wise merge coefficients by minimizing prediction entropy, and RegMean (Jin et al., 2023), which merges each linear layer in closed form from input Gram matrices. RegMean is given the same 64 unlabeled inputs per task that APR sees labeled (a 256-input variant changes its scores by less than 0.02, so the small budget is not what limits it). All families share the same experts, the same replay buffers, the same frozen heads, and the same per-family tuning rule.

The refinement composes with better merges rather than replacing them. On CLIP-8 APR improves *every* merge initialization it is given—task arithmetic, Breadcrumbs, TIES, AdaMerging, RegMean, and the supervised calibrated merge introduced below—and the best results always come from the strongest initializations, never from plain task arithmetic: it lifts AdaMerging from 0.610 to 0.722, RegMean from 0.733 to 0.762, and the calibrated merge from 0.737 to **0.825** mean / +**0.543** worst-task. On GLUE-8 the same pattern holds with one instructive exception analysed below: APR lifts DARE-TIES from 0.301 to 0.610 and the calibrated merge from 0.580 to 0.662, but *harms* the RegMean initialization at every learning rate. The refinement is complementary to good merging rather than an alternative to it—and, when its own two stages are used together (calibrate-then-refine), the entire pipeline consumes only the 64 labeled training examples per task, never touching the evaluation split.

Table 6: Multi-seed replication of the headline configurations (five replay-buffer seeds; mean ± std of mean normalized retention, worst-task in parentheses). Each cell re-runs the best configuration from the corresponding sweep with a freshly sampled replay buffer per seed; TIES is deterministic given the checkpoints and is shown for reference. The random-gate control uses a random mask of the same density as the attribution gate.

Method	CLIP-8 mean ± std (worst ± std)	GLUE-8 mean ± std (worst ± std)
TIES (deterministic)	0.528 (+0.255)	0.302 (0.000)
Ordinary replay GD ← TA	$0.460 \pm .002$ ($-0.200 \pm .007$)	$0.483 \pm .017$ ($-0.048 \pm .058$)
Random gate ← TA	$0.498 \pm .117$ ($-0.564 \pm .888$)	$0.568 \pm .031$ ($+0.030 \pm .043$)
Ungated ← TA	$0.547 \pm .007$ ($-0.068 \pm .016$)	$0.593 \pm .040$ ($+0.085 \pm .096$)
AdaMerging (layer-wise)	$0.595 \pm .037$ ($+0.168 \pm .076$)	—
APR ← TA	$0.605 \pm .008$ ($+0.151 \pm .020$)	$0.542 \pm .062$ ($-0.170 \pm .430$)
APR ← TIES	$0.635 \pm .061$ ($+0.151 \pm .399$)	—
APR ← DARE-TIES	—	$0.620 \pm .021$ ($+0.180 \pm .053$)
APR ← AdaMerging	$\mathbf{0.699 \pm .044}$ ($+0.310 \pm .223$)	—

Error bars, and a single-seed conclusion retracted. Replicating the headline configurations over five replay-buffer seeds (Table 6) mostly confirms the single-seed picture, with one exception that we retract explicitly. In a single-seed sweep, layer-wise AdaMerging (0.610) appeared to edge out APR from the tuned task-arithmetic merge (0.588) on CLIP-8 despite using no labels; at five seeds the ordering is statistically indistinguishable and nominally *reversed* (APR 0.605 ± 0.008 vs. AdaMerging 0.595 ± 0.037), so we treat the two as tied rather than concluding that the label-free method wins. Two related findings survive and are worth keeping. First, AdaMerging is remarkably data-efficient: restricting it from its standard transductive protocol (unlimited unlabeled draws from the evaluation split) to exactly the 64 unlabeled inputs per task that APR sees labeled does not degrade it at all (0.604 → 0.610 mean; +0.165 → +0.219 worst), so its strength is not a data-access artifact. Second, that a label-free method *ties* a labeled one from the same initialization still demands explanation—which the next paragraph provides, and which the calibrate-then-refine experiment below converts into an improvement of our own method.

Why is a label-free method competitive with a labeled one at all? The answer appears to be parameterization rather than information. Inspecting the learned layer-wise coefficients shows that AdaMerging does not merely rescale the merge—the spread of λ_i^l across tensors within a task (0.151) is comparable to its mean (0.258), attention projections are amplified to nearly twice the initial coefficient while embedding and bias contributions are driven toward zero, and 38% of coefficients *increase* relative to initialization. The dominant recoverable error in the CLIP merge is thus a per-tensor mixing error, living in a well-conditioned space of $\sim$1600 coefficients. AdaMerging searches exactly that space; APR searches $\sim 10^8$ coordinates with a step family—coordinate-wise moves *toward* the experts—that cannot express "double this tensor's contribution beyond the merge". Consistent with this reading, the *task-wise* AdaMerging variant, which has only eight degrees of freedom, scores 0.344, worse than a single tuned global coefficient (0.407); the useful knowledge is which tensors to re-mix, not which tasks to weight. This diagnosis makes a concrete prediction: giving the *labeled* pipeline the same low-dimensional degrees of freedom should recover AdaMerging's advantage while avoiding its failure mode on trained heads. The calibrate-then-refine experiment below tests exactly that.

Coefficient tuning corrects an earlier conclusion. The uniform $\lambda_i = 0.3$ used throughout the CLIP experiments above is substantially over-scaled: $\lambda_i = 0.2$ raises the merge from 0.270 / −0.527 to 0.407 / +0.069. As noted in Section 4, this invalidates any reading of Table 3 in which APR is the only method with positive worst-task retention on CLIP-8; tuned task arithmetic, Breadcrumbs, and TIES all achieve it using no data. On GLUE-8, by contrast, $\lambda = 0.3$ is already the interior optimum of the same grid (0.175, **0.320**, 0.239, 0.143 for $\lambda \in \{0.2, 0.3, 0.4, 0.5\}$), so the GLUE comparisons above are unaffected.

Entropy is a good surrogate for accuracy only when the heads are frozen. AdaMerging's usefulness is modality-dependent in the opposite direction to the gate's. On GLUE-8 it drives its unlabeled objective down hard—mean entropy 0.610 → 0.163 for the layer-wise variant—yet reaches only 0.173 mean retention, far *below* the 0.320 of the merge it starts from, and the task-wise variant collapses outright (0.023; worst-task −3.068, MRPC). The explanation is structural. CLIP's heads are frozen zero-shot text embeddings, so confidence can only be increased by producing better image features; RoBERTa's heads are trained classifiers, and the shared encoder can instead be dragged into whatever region makes those classifiers confident, which is not the region that makes them correct. Minimizing entropy then makes the merged model confidently wrong. This is worth stating because it cuts against a natural intuition—that a label-free objective which improves on one benchmark is a safe default—and because it is precisely the failure mode that a *labeled* replay signal, as used here, cannot exhibit.

Calibrate-then-refine: closing the parameterization gap with the same labels. The two observations above—that the dominant recoverable error is per-tensor mixing, and that a labeled objective cannot be confidently wrong—combine into the two-stage instantiation of our method formalized in Section 2.4, which uses nothing beyond its existing replay buffer. *Calibrate*: fit the per-tensor coefficients $\lambda_i^{(b)}$ of Equation (21) by minimizing the labeled replay loss (Equation (22); the supervised twin of AdaMerging, cross-entropy in place of entropy), regularized by an ℓ_2 pull toward the initial coefficient and early-stopped on a held-out quarter of the buffer. *Refine*: run the gated replay refinement (Algorithm 1) from the calibrated merge $\theta_{\text{cal}}^{(0)}$. Calibration fixes the low-dimensional tensor-mixing error; the refinement fixes the residual coordinate-level interference.

The result is the strongest configuration in this study, on both suites, fully inductively. On CLIP-8 the calibrated merge alone reaches 0.737—already ahead of every other merge, labeled or not—and refinement lifts it to **0.825** mean / **+0.543** worst-task (an interior learning-rate peak). On GLUE-8, where AdaMerging's entropy objective collapses (0.173), the *labeled* calibration works (0.580), and refinement lifts it to **0.662** / +0.163. Two anticipated failure modes did not materialize: fitting $\sim 10^3$ coefficients on 64 labeled examples per task did not overfit (the lightest regularization won its sweep, with early stopping doing the remaining work), and the calibration transferred across modalities precisely because correctness and confidence coincide under a labeled objective. We regard calibrate-then-refine as the recommended form of the method: the same 64 labels are spent twice, first on the mixing error in a well-conditioned 10^3-dimensional space, then on the interference error in the full parameter space.

Which ingredient of the sparsification baselines matters. DARE alone reproduces task arithmetic almost exactly (CLIP 0.270 vs. 0.270; all nine sampled cells fall in $[0.249, 0.272]$), as it must: its drop-and-rescale is expectation-preserving, so summed over $\sim 10^8$ coordinates the merge concentrates back onto the unsparsified one. It is therefore reported as a control rather than a competitor, and it isolates *sign election*—not sparsification—as the operative ingredient of

TIES. Pairing the two (DARE-TIES) is scale-sensitive, because DARE's 1/density rescale inflates magnitudes before the trim: at trim density 0.1 it is well behaved, and at 0.2 it collapses MRPC.

The optimal step size depends on the initialization—and boundedness is not usefulness. APR's raw update and its trust region both scale with the expert distance $|v_{i,r}| = |\theta_{i,r} - \theta_r|$, which differs across merge points, and the two roles of that scale are worth separating. The clip binds exactly when $\eta\,|g_{i,r}| > \gamma$—the $|v_{i,r}|$ cancels—so the learning rate controls *what fraction* of coordinates saturate, while the initialization's distance controls *the absolute length* of every saturated move ($\gamma|v_{i,r}|$: the whole way to the expert). From a distant initialization the same η therefore produces the same saturation fraction but vastly larger physical steps: at $\eta = 8$ we measure first-sweep step norms of 0.13–2.96 from the RegMean merge versus 0.035–0.13 from the calibrated merge, a 4–30× difference at identical gate density. In the saturated regime the dynamics remain *bounded*—each task's step still cannot overshoot its own expert, and we verified convergence up to $\eta = 10^6$—but they converge to a coordinate-wise patchwork of the eight experts, overwritten task by task, whose quality is close to the unrefined merge. Boundedness is a tuning-robustness property; usefulness requires steps that are perturbative relative to the initialization's structure. Re-centering the grid accordingly recovers interior optima: APR from RegMean peaks at $\eta = 0.5$ on CLIP-8 ($0.733 \to 0.762$), thirty times below the task-arithmetic optimum. Measuring the geometry confirms the same mechanism behind the earlier TIES failures: relative to task arithmetic, the TIES merge is *closer* to six of the eight GLUE experts but substantially farther from the two large-data ones (QQP, MNLI), so at a fixed η those two tasks take steps 1.6–2.2× larger and trample the fragile MRPC; in every failed cell exactly one fragile task collapses (MRPC on GLUE, Cars on CLIP).

One exception is not a step-size effect and delimits when composition works. On GLUE-8, APR harms the RegMean initialization at *every* learning rate, including deeply perturbative ones ($0.478 \to 0.103$ at $\eta = 0.25$). Two ingredients coincide there: the gate is uninformative on GLUE (no better than random; Table 6), and RegMean's least-squares solution is exactly the point that *deviates* from the task-vector directions to cancel interference—so any movement back toward the experts, however small, undoes the fit. Composition with APR therefore requires (i) a gate that carries signal on the modality, and (ii) an initialization whose structure survives movement toward the experts; task arithmetic, TIES, AdaMerging, and the calibrated merge satisfy (ii), while a regression optimum does not.

Matched labeled budget: the same 64 examples, every method. The comparison so far spans the checkpoint-only and label-free regimes, but the method lives in the small-labeled-data regime, and the sharpest test holds the data fixed and varies only the method. Table 8 gives every labeled method exactly APR's $n = 64$ replay examples per task, drawn from the training split, and—unlike the tables above—selects each method's hyperparameters on the *replay buffer* rather than the evaluation split (the strict protocol of Section 4); we note where this replay-selected number differs from the evaluation-selected one. Against this budget, alternative uses of the same labels are surprisingly weak. Tuning a single global λ recovers the task-arithmetic point (it is already optimal); tuning per-task λ_i by coordinate descent *overfits* the buffer, collapsing to 0.144 on GLUE-8; LM-Cocktail-style loss weighting (Xiao et al., 2024) and Fisher merging (Matena and Raffel, 2022) with a diagonal Fisher estimated from 64 examples are both weak on both suites. Localize-and-Stitch (He et al., 2024), the closest published relative of the attribution gate, is the most instructive comparison. Its *learned* masks—trained on the same replay buffer, with the ℓ_1 sparsity penalty swept over four orders of magnitude and the stitch scale tuned—reach only 0.328 / 0.173, consistently *below* its own data-free magnitude variant (0.520 / 0.415): at this sample size,

learning which coordinates to keep is worse than simply taking the largest ones. The competitive labeled baselines are instead the ones that touch many parameters: the magnitude stitch and the encoder-refinement family. Among single-stage methods on CLIP-8, APR is the best at this budget (0.598, confirmed at 0.605 ± 0.008 over five seeds); on GLUE-8 the encoder-refinement family dominates but the gate is not the decisive ingredient there (0.542 ± 0.062 for APR versus 0.593 ± 0.040 ungated), consistent with the modality-dependence established above. The two-stage calibrate-then-refine tops the table on both suites (0.825 / 0.655 replay-selected), spending the same 64 labels on both stages.

Head-only recalibration exhibits the same modality asymmetry as AdaMerging, and for the same reason: refitting each task head on 64 examples over a frozen merged encoder is the second-strongest method on GLUE-8 (0.613) but catastrophic on CLIP-8 (-0.415, with SUN397 and Cars driven far below the zero-shot floor). A trained GLUE classifier head can absorb a small labeled sample; CLIP's head is a frozen zero-shot anchor, and unfreezing it to fit 64 images destroys the very alignment that makes the merge work. The labeled signal helps through the head only when the head is trainable, which reinforces that APR's gains on CLIP are encoder repair rather than head realignment.

What is still missing. The baseline tiers are now populated (checkpoint-only, label-free with RegMean and AdaMerging, and the matched labeled tier including learned Localize-and-Stitch), and the headline configurations carry error bars. The remaining gaps are protocol-level: the hyperparameter *sweeps* behind Tables 7 and 3 are still single-seed and evaluation-selected, whereas the strict replay-selection protocol is applied only in Table 8; the calibrate-then-refine cells await multi-seed replication; and the shared-output (text-to-text) and decoder-LLM settings promised in the experimental protocol remain the subject of ongoing work. The matched-budget comparison already answers the central alternative explanation—that a small labeled sample would help *any* reasonable method equally: at 64 labeled examples per task, most alternative uses of the same data are markedly worse than the refinement, the strongest single-stage competitors are themselves parameter-space edits of the merge, and the best use of the labels we found is the proposed two-stage one.

5 Expected Contribution

The expected contribution is an attribution-patching perspective on replay-guided model merging. Rather than using attribution scores to build a pre-merge sparsification mask, the method starts from a standard task-arithmetic merge and asks a local intervention question: for task i, which coordinates would reduce the replay loss if the current model moved toward expert i? The answer is the sign of the coordinate-wise product $g_{i,r} v_{i,r}$, which coincides with a trust-region acceptance rule that keeps each step moving toward its expert, so the attribution-patching and replay-refinement views motivate the same procedure. This leads to a simple implementable algorithm: mask out expert directions that are locally loss-increasing, scale the remaining negative-gradient update by distance to the expert, clip each per-task update by a trust-region fraction, and immediately apply that task's update before computing the next task's gradient and gate. Empirically, its strongest form is two-stage: first fit per-tensor merge coefficients on the same labeled replay buffer (a supervised counterpart of AdaMerging that cannot be confidently wrong), then run the gated refinement from that calibrated merge—the two stages address the tensor-level mixing error and the coordinate-level interference error respectively. The proposal also contributes a stricter empirical protocol: fair labeled-replay baselines, an aggregated-update ablation, mask-stability tests,

systematic GLUE pair/subset evaluations, and secondary shared-head or CLIP/ViT benchmarks. If successful, the method would provide a lightweight bridge between checkpoint-only task arithmetic and full multitask fine-tuning with replay buffers, while making clear which gains come from the attribution gate rather than from supervised replay alone.

References

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.

Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.

Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.

Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.

Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Xingrun Xing. LM-Cocktail: Resilient tuning of language models via model merging. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning*, 2017.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.

Table 7: Merge baselines and composition (single seed, $n = 64$, $S = 5$). Mean normalized retention, worst-task in parentheses. Every family shares the same experts, replay buffers, and heads; hyperparameters (λ, density, sparsity, η) are tuned per family on the evaluation split (see Section 4). ‡: standard *transductive* AdaMerging draws unlabeled inputs from the evaluation split; the CLIP rows instead use the matched budget (the same 64 unlabeled inputs per task that APR sees labeled), which *improves* on transductive (0.333 / 0.604 for task/layer wise), and the matched layer-wise merge is the initialization for the last row (making that composition fully inductive). On GLUE both budgets fail alike (transductive 0.023 / 0.173; matched 0.029 / 0.198 for task/layer wise). RegMean uses the matched 64 unlabeled inputs per task (a 256-input variant scores 0.747 / 0.483). The calibrated merge is the supervised per-tensor coefficient fit of the calibrate-then-refine paragraph. ¶: APR harms the RegMean initialization on GLUE-8 at every learning rate (see text); the value shown is its least-bad cell. Multi-seed replications of the headline cells appear in Table 6.

Method	CLIP-8 mean (worst)	GLUE-8 mean (worst)
Checkpoint-only (no data)		
Task arithmetic, $\lambda_i = 0.3$	0.270 (−0.527)	0.320 (−0.175)
Task arithmetic, λ tuned	0.407 (+0.069)	0.320 (−0.175)
DARE (control)	0.270 (−0.524)	0.337 (+0.009)
DARE-TIES	0.364 (−0.319)	0.301 (−0.344)
Model Breadcrumbs	0.431 (+0.079)	0.143 (−0.267)
TIES	0.528 (+0.255)	0.302 (0.000)
Label-free, data-dependent		
AdaMerging (task-wise)	0.344 (−0.239)	0.023‡ (−3.068)
AdaMerging (layer-wise)	0.610 (+0.219)	0.173‡ (0.000)
RegMean	0.733 (+0.443)	0.478 (+0.178)
Labeled replay ($n = 64$)		
Calibrated merge (per-tensor, labeled)	0.737 (+0.390)	0.580 (+0.021)
APR ← task arithmetic	0.588 (0.185)	0.595 (0.108)
APR ← Breadcrumbs	0.606 (0.212)	0.523 (0.095)
APR ← DARE-TIES	0.626 (0.190)	0.610 (0.178)
APR ← TIES	0.653 (0.333)	0.571 (0.165)
APR ← AdaMerging	0.722 (0.429)	0.583 (0.050)
APR ← RegMean	0.762 (0.441)	0.103¶ (−0.062)
APR ← calibrated merge	**0.825** (**0.543**)	**0.662** (**0.163**)

Table 8: Matched labeled-data budget: every method uses the same 64 labeled examples per task (from train), with hyperparameters selected on the replay buffer, not the evaluation split. Mean normalized retention, worst-task in parentheses. §: the evaluation-selected configuration differs from the replay-selected one shown (evaluation-selected values: ungated CLIP 0.536; APR GLUE 0.595; learned Localize-and-Stitch 0.371 / 0.421; calibrated merge CLIP 0.737; APR ← calibrated GLUE 0.662).

Method ($n = 64$ labeled)	CLIP-8	GLUE-8
Global λ tuned on replay	0.270 (−0.53)	0.320 (−0.18)
Per-task λ_i (coordinate descent)	0.377 (−0.02)	0.144 (−1.96)
LM-Cocktail loss-weighted	0.214 (+0.04)	0.074 (−0.03)
Fisher merging (replay Fisher)	0.231 (−0.05)	−0.010 (−1.16)
Head-only recalibration	−0.415 (−4.30)	0.613 (+0.13)
Localize-and-Stitch (learned masks)	0.328§ (−0.10)	0.173§ (−1.10)
Localize-and-Stitch (magnitude, data-free)	0.520 (+0.22)	0.415 (+0.02)
Ordinary replay GD	0.458 (−0.20)	0.504 (−0.02)
Ungated distance-scaled	0.521§ (−0.32)	0.637 (+0.18)
APR (proposed)	0.598 (+0.16)	0.570§ (+0.09)
Calibrated merge (per-tensor, labeled)	0.734§ (+0.36)	0.580 (+0.02)
APR ← calibrated (proposed)	**0.825** (+**0.54**)	**0.655**§ (+0.03)